

index moves right to receive future smaller-than-the-pivot values. As a result, everything to the left of the `i` index is smaller than or equal to the pivot, and everything to the right of the `i` index is greater than the pivot.

The entire process repeats as we recursively call `quicksort()` on the left and right partitions. When we partition these two halves (and then partition the four halves of these two halves with more recursive `quicksort()` calls, and so on), the entire array ends up sorted.

When we run these programs, the output shows the process of sorting the `[0, 7, 6, 3, 1, 2, 5, 4]` list. The rows of periods are meant to help you line up the output when writing the code:

```
quicksort() called on this range: [0, 7, 6, 3, 1, 2, 5, 4]
.....The full list is: [0, 7, 6, 3, 1, 2, 5, 4]
.....The pivot is: 4
....After swapping, the range is: [0, 3, 1, 2, 4, 7, 5, 6]
Recursively calling quicksort on: [0, 3, 1, 2] and [7, 5, 6]

quicksort() called on this range: [0, 3, 1, 2]
.....The full list is: [0, 3, 1, 2, 4, 7, 5, 6]
.....The pivot is: 2
....After swapping, the range is: [0, 1, 2, 3]
Recursively calling quicksort on: [0, 1] and [3]

quicksort() called on this range: [0, 1]
.....The full list is: [0, 1, 2, 3, 4, 7, 5, 6]
.....The pivot is: 1
....After swapping, the range is: [0, 1]
Recursively calling quicksort on: [0] and []

quicksort() called on this range: [0]
.....The full list is: [0, 1, 2, 3, 4, 7, 5, 6]

quicksort() called on this range: []
.....The full list is: [0, 1, 2, 3, 4, 7, 5, 6]

quicksort() called on this range: [3]
.....The full list is: [0, 1, 2, 3, 4, 7, 5, 6]

quicksort() called on this range: [7, 5, 6]
.....The full list is: [0, 1, 2, 3, 4, 7, 5, 6]
.....The pivot is: 6
```